PROYECTO INTEGRADOR - PROGRAMACIÓN I 2026

Sistema de gestión de reparto urbano

Enunciado de los Entregables 1 y 2

1. Presentación y modalidad

Este proyecto integra todos los contenidos de la materia: programación orientada a objetos (clases, herencia, clases y métodos abstractos, redefinición, sobrecarga, polimorfismo), manejo de excepciones, estructuras de datos estáticas (arreglos) y dinámicas (listas, pilas, colas y árboles, con las clases de java.util) y algoritmos de búsqueda, ordenamiento e intercalación. Se desarrolla en dos entregables sobre un mismo sistema: el Entregable 2 es la evolución del Entregable 1. Los grafos y la complejidad algorítmica se trabajan y se evalúan en la parte teórica de la materia: no se implementan en el proyecto.

- **Grupos** de 2 o 3 integrantes. (vemos otras propuestas)
- **Lenguaje y entorno:** Java, aplicación de consola (línea de comandos). IDE a elección (VSCode, IntelliJ IDEA u otro).
- **Repositorio:** ambos entregables se presentan en un repositorio de GitHub con README y código. Deben verse commits de todos los integrantes.
- **Exposición oral:** cada entregable se defiende oralmente. El grupo presenta lo que hizo, muestra el sistema funcionando y cada integrante responde preguntas sobre cualquier parte del código.
- **Calificación:** cada entregable se califica de 0 a 100 según la rúbrica incluida en este documento.
- **Uso de IA:** está permitido (autocompletado, corrección de errores, documentación), pero debe declararse en el README y cada integrante debe poder explicar cualquier línea del código entregado.

2. El problema:

2.1 Descripción

CityLink es una empresa de reparto de paquetes de una ciudad. Los clientes hacen pedidos de envío desde un **depósito central** hacia distintos **puntos de la ciudad**. La empresa cuenta con repartidores, cada uno con un vehículo (bicicleta, moto o camioneta), y necesita un sistema por consola que le permita registrar clientes, repartidores y pedidos, asignar los pedidos a quien pueda transportarlos, seguir su estado y calcular costos y tiempos de entrega.

En el Entregable 1 se construye el modelo de objetos del sistema con manejo de errores. En el Entregable 2 se reemplaza el almacenamiento por las estructuras dinámicas de java.util y se incorporan algoritmos de búsqueda, ordenamiento e intercalación.

2.2 Entidades y datos

Entidad	Datos
Cliente	Código (entero único), nombre, teléfono y zona de la ciudad.
Pedido	Código único, cliente, destino (zona o punto de la ciudad), peso en kg, distancia en km, fecha y hora de ingreso, estado y tipo (normal, urgente o frágil).
Repartidor	Legajo, nombre, vehículo asignado y pedidos a su cargo.
Vehículo	Patente, capacidad en kg, velocidad promedio en km/h y costo por km.

2.3 Vehículos de referencia

Vehículo	Capacidad	Velocidad	Costo por km	Restricción y tiempo extra
Bicicleta	10 kg	15 km/h	$50	No puede transportar pedidos frágiles. Sin tiempo extra.
Moto	30 kg	40 km/h	$120	5 minutos extra por estacionamiento.
Camioneta	500 kg	30 km/h	$300	10 minutos extra por carga y descarga.

Los valores son orientativos: el grupo puede ajustarlos siempre que los documente en el README.

2.4 Reglas de negocio

- **Costo del envío** = costo base de $1000 + (distancia en km × costo por km del vehículo). Un pedido **urgente** suma un 50% sobre ese total. Un pedido **frágil** suma un adicional fijo de $800. Un pedido **normal** no tiene recargo.
- **Tiempo estimado** (en minutos) = (distancia / velocidad) × 60 + tiempo extra del vehículo.
- Un pedido no puede asignarse a un vehículo si su peso supera la capacidad, ni un pedido frágil a una bicicleta.
- **Estados del pedido:** PENDIENTE → EN_CAMINO → ENTREGADO. Un pedido puede pasar a CANCELADO solo desde PENDIENTE. Cualquier otra transición es inválida.
- El peso y la distancia deben ser mayores que cero, el destino no puede estar vacío, el código de pedido no puede repetirse y el cliente debe existir.
- **Comisión del repartidor** = 10% del importe de los pedidos que entregó.
- La distancia de cada pedido se ingresa como dato al registrarlo.

3. Entregable 1: objetos y manejo de errores

3.1 Objetivo

Diseñar e implementar el modelo de objetos de **CityLink** aplicando encapsulamiento, herencia, abstracción, interfaces, redefinición y sobrecarga, polimorfismo de clases y de métodos, y un manejo de excepciones completo. Los datos se almacenan en **arreglos** (estructuras estáticas).

3.2 Clases requeridas

Clase	Tipo	Contenido mínimo
Cliente	Concreta	Atributos privados (código, nombre, teléfono, zona), constructor vacío y completo, getters y setters con validación, toString.

Clase	Tipo	Contenido mínimo
Pedido	Abstracta	Código, cliente, destino, pesoKg, distanciaKm, fechaHora y estado. Método abstracto calcularCosto(Vehiculo). Método concreto cambiarEstado que valida las transiciones.
PedidoNormal, PedidoUrgente, PedidoFragil	Concretas	Redefinen calcularCosto aplicando el recargo que corresponde a cada tipo.
Vehiculo	Abstracta	Patente, capacidadKg, velocidadKmH y costoPorKm. Método abstracto calcularTiempoEntrega(double km). Método concreto puedeTransportar(Pedido).
Bicicleta, Moto, Camioneta	Concretas	Redefinen calcularTiempoEntrega con su tiempo extra. Bicicleta además redefine puedeTransportar para rechazar pedidos frágiles.
Repartidor	Concreta	Legajo, nombre, vehículo y arreglo de pedidos asignados. Métodos asignarPedido y registrarEntrega.
Rastreable	Interfaz	obtenerEstadoActual(). La implementan Pedido y Repartidor.
Facturable	Interfaz	calcularImporte(). La implementan Pedido (costo del envío) y Repartidor (comisión).
CityLink (gestor) y Main	Concretas	El gestor contiene los arreglos y las operaciones del sistema. Main solo contiene el menú y la lectura por teclado.

3.3 Conceptos de POO que se deben evidenciar

- **Encapsulamiento:** todos los atributos son privados, con acceso por getters y setters. Los setters validan los datos.
- **Herencia y abstracción:** al menos dos jerarquías (Pedido y Vehiculo) con clase abstracta en la raíz, métodos abstractos y subclases que los implementan.
- **Interfaces y herencia múltiple:** Java no permite que una clase herede de dos clases, pero sí implementar varias interfaces. Pedido debe extender de una clase (o ser raíz de su jerarquía) e implementar Rastreable y Facturable a la vez.
- **Redefinición:** calcularCosto, calcularTiempoEntrega y puedeTransportar, con @Override.
- **Sobrecarga:** como mínimo buscarCliente(int codigo) y buscarCliente(String nombre), asignarPedido(Pedido) y asignarPedido(Pedido, Vehiculo), y constructores con distintos parámetros.
- **Polimorfismo de clases:** el sistema recorre un arreglo de Pedido que contiene los tres subtipos y calcula el costo total llamando a calcularCosto, sin usar instanceof ni switch por tipo para decidir el comportamiento. Lo mismo con un arreglo de Vehiculo y calcularTiempoEntrega.
- **Polimorfismo de métodos:** mediante la sobrecarga y la redefinición descritas arriba.

3.4 Excepciones

Excepción	Tipo	Cuándo se lanza
ClienteNoEncontradoException	Checked (extends Exception)	Al buscar un cliente que no existe.
PedidoInvalidoException	Checked	Peso o distancia no positivos, destino vacío o código duplicado.

Excepción	Tipo	Cuándo se lanza
CapacidadExcedidaException	Checked	El peso supera la capacidad del vehículo o se intenta asignar un pedido frágil a una bicicleta.
AlmacenamientoLlenoException	Checked	Se intenta agregar a un arreglo que ya está lleno.
EstadoInvalidoException	Unchecked (extends RuntimeException)	Transición de estado no permitida.

- Las excepciones se **lanzan** (throw) en las clases del modelo y del gestor, se **propagan** con throws y se **capturan** en el menú, que informa el error al usuario y sigue funcionando.
- Al menos una excepción debe propagarse a través de dos o más niveles de llamadas, y debe haber al menos un bloque finally.
- Los datos inválidos por teclado (por ejemplo, texto donde se espera un número) deben capturarse sin que el programa termine.
- **Tipos de errores:** en la exposición, cada grupo explica la diferencia entre error de compilación, de ejecución y lógico, con un ejemplo real que encontró durante el desarrollo.

3.5 Almacenamiento con arreglos

- Clientes (capacidad 50), pedidos (100) y repartidores (10), con constantes para las capacidades y un contador de elementos cargados.
- Eliminar un elemento debe desplazar los siguientes para no dejar huecos.

3.6 Funcionalidades del menú

1. Registrar cliente.
2. Registrar repartidor con su vehículo.
3. Registrar pedido (normal, urgente o frágil).
4. Asignar un pedido a un repartidor, verificando capacidad y restricciones.
5. Cambiar el estado de un pedido (en camino, entregado, cancelado).
6. Buscar un cliente por código o por nombre.
7. Listar pedidos con su costo y tiempo estimado (polimorfismo).
8. Consultar el estado de un pedido o de un repartidor (Rastreable).
9. Ver el importe total de los pedidos y la comisión de cada repartidor (Facturable).
10. Salir.

El sistema debe iniciar con **datos de prueba precargados**: al menos 5 clientes, 3 repartidores (uno por tipo de vehículo) y 8 pedidos que incluyan los tres tipos.

3.7 Condiciones técnicas

- **Paquetes:** las clases se organizan en carpetas llamadas paquetes. El paquete **modelo** contiene las clases del negocio (Cliente, Pedido, Vehiculo, Repartidor, sus subclases y las interfaces), el paquete **excepciones** contiene las excepciones propias y el paquete **app** contiene Main y el menú. Cada archivo indica su paquete en la primera línea (por ejemplo, package modelo;) y usa import para acceder a las clases de otro paquete.
- Convenciones de nombres de Java y comentarios Javadoc en las clases y los métodos públicos.

- README con integrantes, instrucciones para compilar y ejecutar, diagrama de clases (imagen) y declaración de uso de IA (qué herramienta, para qué y cómo se verificó el resultado).
- Uso de Git durante todo el desarrollo, con commits descriptivos de todos los integrantes.

4. Entregable 2: estructuras dinámicas de Java y algoritmos

4.1 Objetivo y punto de partida

Se parte del sistema del Entregable 1 y se reemplazan los arreglos por las **estructuras dinámicas que trae Java** (paquete java.util): lista, pila, cola y árbol. Además se escriben algoritmos de búsqueda, ordenamiento e intercalación. Se debe **conservar** el diseño orientado a objetos, el polimorfismo y el manejo de excepciones del Entregable 1.

4.2 Estructuras dinámicas de java.util

- Se usan las clases de la librería: **no** se implementan a mano. Sí deben entender cómo funcionan por dentro, y eso se evalúa en la exposición oral.
- Estas clases se usan indicando entre < > el tipo de objeto que guardan: ArrayList<Pedido> es una lista que guarda objetos Pedido, y TreeMap<Integer, Cliente> guarda clientes ordenados por un código entero.

Estructura	Clase de Java	Métodos principales	Uso en CityLink
Lista	ArrayList o LinkedList (interfaz List)	add, add(indice, elemento), get, remove, contains, indexOf, size, isEmpty, recorrido con for-each	Repartidores, pedidos asignados a cada repartidor, historial de entregas y pedidos de cada turno.
Pila	ArrayDeque (interfaz Deque)	push, pop, peek, isEmpty, size	Deshacer: cada operación del operador (alta de pedido, asignación, cambio de estado) se apila y “Deshacer” revierte la última.
Cola	ArrayDeque o LinkedList (interfaz Queue)	offer, poll, peek, isEmpty, size	Pedidos pendientes en orden de llegada. Hay una cola de urgentes y otra de normales: se atiende primero la de urgentes y, dentro de cada una, por orden de llegada.
Árbol de búsqueda	TreeMap (o TreeSet)	put, get, remove, containsKey, firstKey, lastKey, recorrido en orden	Clientes por código. El listado ordenado sale del recorrido en orden del árbol.

- Si el sistema necesita sacar de una estructura vacía (por ejemplo, atender un pedido cuando no hay pendientes), debe informarlo al usuario sin cortar el programa: con una excepción propia (por ejemplo, SinPedidosPendientesException) o capturando la que lanza Java.

4.3 Algoritmos (escritos por el grupo)

Estos algoritmos se **escriben a mano**, sin usar Collections.sort, List.sort ni binarySearch de Java, porque el objetivo es aprender cómo funcionan. Trabajan sobre las listas o arreglos del sistema.

- **Búsqueda lineal:** buscar un pedido por código en una lista y un cliente por nombre.
- **Búsqueda binaria:** buscar un pedido por código sobre una lista o arreglo ordenado (al menos una versión, iterativa o recursiva).
- **Ordenamiento:** al menos dos algoritmos distintos (por ejemplo, burbuja o inserción, y quicksort o mergesort). Deben permitir ordenar los pedidos por costo, por distancia o por fecha, eligiendo el criterio (con Comparator o con un parámetro).
- **Intercalación:** combinar en un único listado ordenado dos listados que ya están ordenados (por ejemplo, los pedidos del turno mañana y del turno tarde), con un solo recorrido y sin volver a ordenar.

4.4 Funcionalidades del menú

1. Gestión de clientes: alta, baja y búsqueda por código (árbol) y listado ordenado por código.
2. Gestión de repartidores (lista).
3. Registrar un pedido: lo encola en la cola que corresponda y registra la operación en la pila.
4. Atender el siguiente pedido: lo saca de la cola (urgentes primero), lo asigna a un repartidor con capacidad suficiente y actualiza su estado.
5. Registrar una entrega y agregarla al historial del repartidor.
6. Deshacer la última operación (pila).
7. Ordenar y listar pedidos eligiendo el criterio y el algoritmo.
8. Buscar un pedido por código (lineal o binaria).
9. Intercalar los pedidos de dos turnos.
10. Salir.

4.5 Condiciones técnicas

- Paquetes: modelo, excepciones, algoritmos (clases con la búsqueda, el ordenamiento y la intercalación) y app.
- Pruebas documentadas en el README: casos normales y casos borde (estructura vacía, un solo elemento, elemento inexistente).
- Git con commits descriptivos de todos los integrantes.
- README actualizado con diagrama de clases, instrucciones de ejecución y declaración de uso de IA (qué herramienta, para qué y cómo se verificó el resultado).